

Redis Data Structures

IEM 4723 Information Systems Design

Dr. Paritosh Ramanan

School of Industrial Engineering and Management
Oklahoma State University

Fall 2026

Adapted from DigitalOcean, How To Manage a Redis Database. Commands are shown using redis-py rather than the redis-cli form used in the book.

- 1 Running Redis
- 2 Redis Data Structures
- 3 Expiring Keys

Running Redis

- Redis runs in a container, in the same way MySQL has throughout this course. The tutorial stack lives in `docker_tutorial/redis-dev/` and defines two services, `redis` itself and a jupyter notebook server to drive it from Python.

```
docker compose up -d --force-recreate
```

- As before, a single service can be started on its own if one of them fails to come up.

```
docker compose up redis -d
```

- Redis listens on port 6379 by default, in the same way MySQL listens on 3306.

Connecting from Python

We use the `redis` package for Python, usually called `redis-py`, in the same spirit as `pymysql` earlier in the course.

```
import redis

r = redis.Redis(
    host="redis", # the service name in docker-compose.yml
    port=6379,
    decode_responses=True # see the note below
)
r.ping()
```

- `decode_responses=True` matters more than it looks. Without it every value comes back as raw bytes, so a stored name reads as `b'Felton'` rather than `'Felton'`. Set it once here and every later result is an ordinary Python string.
- Notice there is no cursor and no connection object to manage separately. In Redis the client itself is the interface.

Keys and Values

The whole model is a key and the value stored against it.

```
r.set("customer:C25:city", "HERSHEY")  
r.get("customer:C25:city") # 'HERSHEY'
```

```
r.exists("customer:C25:city") # 1  
r.delete("customer:C25:city") # 1  
r.exists("customer:C25:city") # 0
```

- There is no table, no schema, and no column list. Nothing had to be created before that first write.
- The colon in the key name is a convention rather than a rule. Redis sees one flat key, but naming keys this way keeps a large keyspace readable.

Working With the Keyspace

```
r.keys("customer:*") # every key matching the pattern  
r.dbsize() # how many keys are in this database  
r.flushdb() # delete every key in this database
```

- A Redis server holds several numbered databases, and a client works in one of them at a time. The database is chosen when connecting, with `redis.Redis(..., db=1)`, rather than by a separate statement.
- Treat `keys()` as a teaching and debugging tool. It scans the entire keyspace, so on a production server with millions of keys it is a genuinely expensive call.
- `flushdb()` empties the database without asking for confirmation, in the same spirit as a `DELETE` with no `WHERE` clause.

Redis Data Structures

More Than a Dictionary

- In Lecture 11 every value we stored was a single string. If that were all Redis offered, it would be a dictionary that happens to live on the network.
- What makes Redis useful is that a value can itself be a data structure, and that Redis understands the structure rather than treating it as an opaque blob.
- That matters because the operations happen on the server. Adding one item to a list of a million does not require fetching the list, changing it in Python, and writing it back.
- We cover five structures. Each exists because some ordinary problem is awkward without it.

Strings

The simplest value, and the one from last lecture. What makes it interesting is that Redis knows when a string holds a number.

```
r.set("page:views", 0)
r.incr("page:views") # 1
r.incr("page:views") # 2
r.incrby("page:views", 10) # 12
r.decr("page:views") # 11
```

- Each of these is atomic. Two clients incrementing at the same moment both count, which is exactly the problem that a read, add one, write back sequence gets wrong under concurrency.
- Doing the same thing in SQL means a transaction and a row lock. Here it is one call.

Hashes

A hash stores field and value pairs under a single key, which makes it the closest thing Redis has to a row in a relational table.

```
r.hset("customer:C25", mapping={  
    "first_name": "Bob",  
    "last_name": "Felton",  
    "city": "HERSHEY",  
})
```

```
r.hget("customer:C25", "city") # 'HERSHEY'  
r.hgetall("customer:C25") # the whole record as a dict  
r.keys("customer:C25") # the field names
```

- Compare this to the customer table from the MySQL work. The same record, with no schema declared ahead of time, and no requirement that the next customer carry the same fields.
- That freedom is also the cost. Nothing stops a typo creating a field named `citty`, which a relational schema would have rejected outright.

Lists

A list holds an ordered sequence of strings, and can be pushed or popped at either end.

```
r.rpush("orders:pending", "16642826", "16644074")  
r.lpush("orders:pending", "16639375") # on the front
```

```
r.lrange("orders:pending", 0, -1) # every element  
r.llen("orders:pending") # how many  
r.lpop("orders:pending") # take from the front
```

- Pushing at one end and popping at the other gives a queue, which is how Redis lists are most often used in practice, for job queues and work backlogs.
- Index -1 means the last element, exactly as in Python, so $0, -1$ is the whole list.

Sets

A set is an unordered collection with no duplicates, and the question it answers quickly is whether something is a member.

```
r.sadd("items:shirts", "14", "18", "25", "38", "71")  
r.sadd("items:location:A9309", "14", "18", "25", "38",  
      "71")
```

```
r.sismember("items:shirts", "25") # 1  
r.scard("items:shirts") # 5  
r.smembers("items:shirts") # the whole set
```

```
r.sinter("items:shirts", "items:location:A9309")
```

- Adding a member that is already present changes nothing, so uniqueness is free rather than something to enforce.
- `sinter`, along with `sunion` and `sdiff`, performs the set operations on the server. That intersection is the kind of question that needs a join in SQL.

Sorted Sets

A sorted set is a set where every member also carries a numeric score, and Redis keeps the members ordered by it.

```
r.zadd("items:by_sales", {  
    "71": 6672, "121332-0036": 2704, "25": 1016,  
})
```

```
r.zrevrange("items:by_sales", 0, 2, withscores=True)  
r.zscore("items:by_sales", "71") # 6672.0  
r.zrank("items:by_sales", "25") # position, lowest first
```

- Note the argument order. In redis-py the members and scores go in as a dictionary, which is the reverse of the ZADD key score member form used by the command line and by the textbook.
- The ordering is maintained as items are inserted, so asking for the top ten is immediate and does not sort anything at read time. This is the structure with no clean relational equivalent.

Choosing a Structure

Structure	Holds	Reach for it when
String	One value	Caching a value, counting
Hash	Field and value pairs	Storing a record or object
List	Ordered, duplicates allowed	Queues, recent activity
Set	Unordered, unique	Membership, set operations
Sorted set	Unique, ordered by score	Rankings, leaderboards, anything top N

- The structure is chosen when the data is written, not when it is queried. That is the opposite of the relational habit, where one schema serves whatever query arrives later.
- So the design question in Redis is not how to model the data, but how it will be read.

Expiring Keys

Keys That Delete Themselves

Any key can be given a lifetime, after which Redis removes it without being asked.

```
r.set("session:abc123", "C25")  
r.expire("session:abc123", 60) # seconds
```

```
r.ttl("session:abc123") # seconds remaining  
r.persist("session:abc123") # cancel the expiry
```

```
r.set("session:abc123", "C25", ex=60) # set and expire  
together
```

- `ttl` returns `-1` when a key exists with no expiry, and `-2` when the key is already gone. Those two values are worth remembering, because they are easy to mistake for an error.
- Nothing in the SQL half of this course does this. Expiring a row in MySQL means a scheduled job that goes looking for stale rows and deletes them.

Redis in Front of a Database

This is the most common reason a team runs Redis alongside a relational database.

```
key = "report:monthly_sales"
cached = r.get(key)

if cached is None:
    cached = run_expensive_mysql_query() # slow path
    r.set(key, cached, ex=300) # keep for 5 minutes

use(cached)
```

- The first request pays for the slow query. Every request for the next five minutes is answered from memory, after which the key expires and the cycle repeats.
- This is the CAP discussion from Lecture 11 made concrete. The cached answer may be up to five minutes stale, a deliberate trade of consistency for speed.
- Choosing that window is a design decision. Five minutes may suit a sales dashboard and be completely wrong for an inventory count.

What We Have Not Covered

- **Transactions.** Redis can group commands so they run together, through MULTI and EXEC, with semantics that differ from the SQL transactions discussed earlier.
- **Persistence.** Redis holds data in memory, and offers snapshotting and an append-only log to survive a restart. Which to use is a real operational decision.
- **Replication and clustering.** How Redis scales across machines, and where the CAP trade-offs from Lecture 11 reappear in a concrete form.
- **Other data types.** Streams, bitmaps, geospatial indexes, and HyperLogLog all exist and all solve particular problems.
- Each of these is covered in the DigitalOcean book listed on the syllabus, and all of it is reachable from what we have done here.